

数据的容器

数组与线性存储 · 12题 · C++ & Python 双语对照

本章进入数据结构的核心——数组。12道题目覆盖一维数组的遍历、替换、填充、筛选、翻转、递推（斐波那契）、最值查找、二维数组的行列操作，以及质数判定的算法优化。你将掌握Python的列表推导、enumerate、切片翻转，以及C++的vector/array操作。

NQ043 数组替换 AcWing 737

小鲁学习了数组。给定一个长度为10的数组，将其中所有小于等于0的元素替换为1，然后输出整个数组。

输入 10行，每行一个整数。

输出 10行，每行" $X[i] = Y$ "， i 从0到9， Y 为替换后的值。

范围 $-10^9 \leq \text{值} \leq 10^9$

输入

```
0
-5
63
-8
0
...
```

输出

```
X[0] = 1
X[1] = 1
X[2] = 63
...
```

思路 遍历数组，如果元素 ≤ 0 则替换为1。用下标循环或for-each+索引。C++用for(int i=0;i<10;i++)，Python用enumerate()同时获取索引和值。

技巧 Python的enumerate(arr)返回(index, value)元组。C++中arr[i]访问元素。数组索引从0开始。

C++

```
#include <iostream>
using namespace std;
int main() {
    int x;
    for (int i = 0; i < 10; i++) {
        cin >> x;
        if (x <= 0) x = 1;
        cout << "X[" << i << "] = " << x << end
    }
    return 0;
}
```

Python

```
for i in range(10):
    x = int(input())
    if x <= 0: x = 1
    print(f"X[{i}] = {x}")
```

NQ044 数组填充 AcWing 738

给定一个整数 V ，构造一个长度为10的数组，第0个元素是 V ，此后每个元素是前一个的两倍。

输入 一个整数 V 。

输出 10行，每行" $N[i] = X$ "。

范围 $-10^4 \leq V \leq 10^4$

输入

1

输出

$N[0] = 1$

$N[1] = 2$

$N[2] = 4$

...

思路 递推生成数组： $arr[0]=V$, $arr[i]=arr[i-1]*2$ 。Python用列表推导或循环。C++用vector或直接输出。

技巧 Python的列表推导： $[v*(2**i) \text{ for } i \text{ in range}(10)]$ 直接用位移运算。C++中 $1<$

C++

```
#include <iostream>
using namespace std;
int main() {
    int v;
    cin >> v;
    for (int i = 0; i < 10; i++, v *= 2)
        cout << "N[" << i << "] = " << v << endl;
    return 0;
}
```

Python

```
v = int(input())
for i in range(10):
    print(f"N[{i}] = {v}")
    v *= 2
```

NQ045 数组选择 AcWing 739

读取100个浮点数。对于每个小于等于10的数，按格式输出它在数组中的位置和值。

输入 100行（或一行多个），每行一个浮点数。

输出 对每个 ≤ 10 的数，输出" $A[i] = X$ "（保留1位小数）。

范围 $-10^6 \leq \text{值} \leq 10^6$

输入

0

-5

63

...

输出

$A[0] = 0.0$

$A[1] = -5.0$

...

思路 遍历100次读入，条件判断 ≤ 10 则输出。Python无需存储全部数据，边读边输出即可，节省内存。C++同样。

技巧 流式处理：不需要把100个数都存下来，读一个判断一个输出一个。这种思维在处理大数据时很重要。

C++

```
#include <cstdio>
int main() {
    for (int i = 0; i < 100; i++) {
        double x;
        scanf("%lf", &x);
        if (x <= 10) printf("A[%d] = %.1f\n",
i, x);
    }
    return 0;
}
```

Python

```
for i in range(100):
    x = float(input())
    if x <= 10:
        print(f"A[{i}] = {x:.1f}")
```

NQ046 数组中的行 AcWing 743

给定一个 12×12 的二维数组。读取行号L和操作类型T（'S'求和或'M'求平均），计算第L行所有元素的和或平均值。

输入

第一行L(0-11)。第二行T('S'或'M')。接下来144个浮点数（ 12×12 矩阵）。

输出

结果保留1位小数。

范围

矩阵元素 -10^6 到 10^6

输入

2
S
(144个浮点数...)

输出

(第2行之和, 1位小数)

思路 双层循环遍历矩阵，遇到目标行L时累加。如果是'M'求平均则除以12。

技巧 二维数组读入：外层for i in range(12)，内层for j in range(12)。Python用嵌套列表推导生成矩阵。

C++

```
#include <cstdio>
int main() {
    int l; char t;
    scanf("%d %c", &l, &t);
    double sum = 0, x;
    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++) {
            scanf("%lf", &x);
            if (i == l) sum += x;
        }
    printf("%.1lf\n", t == 'S' ? sum : sum / 12);
    return 0;
}
```

Python

```
l = int(input())
t = input().strip()
sum_val = 0
for i in range(12):
    for j in range(12):
        x = float(input())
        if i == l: sum_val += x
print(f"{sum_val if t == 'S' else sum_val / 12:.1f}")
```

NQ047 数组变换 AcWing 740

给定一个20个整数的数组。将数组前后对称位置互换（第0个和第19个交换，第1个和第18个交换...），输出变换后的数组。

输入

20个整数。

输出

20行，每行" $N[i] = X$ "。

范围

 $-10^9 \leq \text{值} \leq 10^9$

输入

```
0
1
2
...
19
```

输出

```
N[0] = 19
N[1] = 18
...
```

思路 对称交换： $\text{arr}[i] \leftrightarrow \text{arr}[19-i]$ ，只需循环10次（ i 从0到9）。Python用切片 $\text{arr}[::-1]$ 一行翻转。

技巧 Python的 $\text{arr}[::-1]$ 是最简洁的数组翻转方式。C++用 $\text{reverse}()$ 或手写交换。

C++

```
#include <iostream>
using namespace std;
int main() {
    int arr[20];
    for (int i = 0; i < 20; i++) cin >> arr[i];
    for (int i = 0; i < 10; i++) swap(arr[i], arr[19 - i]);
    for (int i = 0; i < 20; i++)
        cout << "N[" << i << "] = " << arr[i] << endl;
    return 0;
}
```

Python

```
arr = [int(input()) for _ in range(20)]
arr.reverse()
for i, v in enumerate(arr):
    print(f"N[{i}] = {v}")
```

NQ048 斐波那契数列 AcWing 741

小鲁在学数列。斐波那契数列的前两项是0和1，之后每一项都是前两项之和。给定N，输出前N项。

输入 一个整数N ($1 \leq N \leq 60$)。

输出 一行，空格隔开的N个整数。

范围 $1 \leq N \leq 60$ (结果可能超过32位int范围!)

输入

5

输出

0 1 1 2 3

思路 递推: $a=0, b=1$; for i in range(N): print(a); $a, b = b, a+b$ 。注意用long long (C++) 避免溢出, Python 自动大整数。

技巧 Python的 $a, b = b, a+b$ 是递推的标准写法。C++注意用long long (64位), $N=60$ 时结果约 1.5×10^{12} 超出32位int。这是DP思想的萌芽——用前面算出的结果推导后面。

C++

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    long long a = 0, b = 1;
    for (int i = 0; i < n; i++) {
        if (i) cout << " ";
        cout << a;
        long long t = a + b; a = b; b = t;
    }
    cout << endl;
    return 0;
}
```

Python

```
n = int(input())
a, b = 0, 1
result = []
for _ in range(n):
    result.append(str(a))
    a, b = b, a + b
print(' '.join(result))
```

NQ049 最小数和它的位置 AcWing 742

给定N和一个包含N个整数的数组，找出数组中的最小值及其位置（如果有多个最小值，输出第一个的位置）。

输入 第一行N。第二行N个整数。

输出 第一行"Menor valor: X"。第二行"Posicao: Y" (位置从0开始)。

范围 $1 \leq N \leq 1000$

输入

10
1 2 3 4 -5 6 7 8 9 10

输出

Menor valor: -5
Posicao: 4

思路 遍历数组维护最小值和位置。Python用`min(arr)`和`arr.index(min_val)`一行搞定，但手写循环理解查找逻辑。

技巧 Python的`enumerate()`同时获得索引和值。`min(arr)`找最小值，`arr.index(v)`找位置。

C++

```
#include <iostream>
using namespace std;
int main() {
    int n, x, mn, pos = 0;
    cin >> n;
    for (int i = 0; i < n; i++) {
        cin >> x;
        if (i == 0 || x < mn) { mn = x; pos = i; }
    }
    cout << "Menor valor: " << mn << endl;
    cout << "Posicao: " << pos << endl;
    return 0;
}
```

Python

```
n = int(input())
arr = list(map(int, input().split()))
mn = min(arr)
print(f"Menor valor: {mn}")
print(f"Posicao: {arr.index(mn)}")
```

NQ050 数组中的列 AcWing 744

与NQ046类似，但这次操作的是列。给定列号C和操作类型T，计算12×12矩阵第C列的和或平均值。

输入 第一行列号C。第二行操作类型T。接下来144个浮点数。

输出 结果保留1位小数。

范围 同NQ046

输入

2
S
(144个浮点数...)

输出

(第2列之和, 1位小数)

思路 与行操作对称：当内层循环到目标列`j==C`时累加。其他逻辑相同。

技巧 行操作和列操作的双重循环结构完全相同，只在判断条件上差一个字母 (`i==L` vs `j==C`)。注意对比理解。

C++

```
#include <cstdio>
int main() {
    int c; char t;
    scanf("%d %c", &c, &t);
    double sum = 0, x;
    for (int i = 0; i < 12; i++)
        for (int j = 0; j < 12; j++) {
            scanf("%lf", &x);
            if (j == c) sum += x;
        }
    printf("%.1lf\n", t == 'S' ? sum : sum / 12);
    return 0;
}
```

Python

```
c = int(input())
t = input().strip()
sum_val = 0
for i in range(12):
    for j in range(12):
        x = float(input())
        if j == c: sum_val += x
print(f"{sum_val if t == 'S' else sum_val / 12:.1f}")
```

NQ051 简单斐波那契 AcWing 717

计算斐波那契数列的第N项。N从0开始：F(0)=0, F(1)=1, F(n)=F(n-1)+F(n-2)。

输入 一个整数N。

输出 第N项的值。

范围 $1 \leq N \leq 60$

输入

4

输出

3

思路 与NQ048类似，但只输出第N项。递推直到第N项。注意N=0的情况。

技巧 Python的a,b=b,a+b在循环中递推。只输出最后一项，无需存储中间结果。

C++

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    long long a = 0, b = 1;
    if (n == 0) { cout << 0 << endl; return 0; }
    for (int i = 1; i < n; i++) {
        long long t = a + b; a = b; b = t;
    }
    cout << b << endl;
    return 0;
}
```

Python

```
n = int(input())
a, b = 0, 1
for _ in range(n):
    a, b = b, a + b
print(a)
```

NQ052 数字序列和它的和 AcWing 722

对于每一对输入的正整数M和N（M

输入 多行，每行两个整数M和N。以M≤0或N≤0结束。

输出 对每对M,N，输出一行所有整数（空格隔开）和"Sum=X"。

范围 M,N ≤ 100

输入	输出
5 10	5 6 7 8 9 10 Sum=45
2 3	2 3 Sum=5
0 0	

思路 while循环读取，遇到非正数break。对每对M,N确保M≤N（必要时交换），然后for循环输出并累加。

技巧 Python的range(m,n+1)生成连续整数。用join(map(str,...))拼接输出整洁。注意先交换保证M≤N。

C++

```
#include <iostream>
using namespace std;
int main() {
    int m, n;
    while (cin >> m >> n, m > 0 && n > 0) {
        if (m > n) swap(m, n);
        int sum = 0;
        for (int i = m; i <= n; i++) {
            cout << i << " ";
            sum += i;
        }
        cout << "Sum=" << sum << endl;
    }
    return 0;
}
```

Python

```
while True:
    m, n = map(int, input().split())
    if m <= 0 or n <= 0: break
    if m > n: m, n = n, m
    nums = range(m, n + 1)
    print(' '.join(map(str, nums)), f"Sum={sum(nums)}")
```

NQ053 完全数 AcWing 725

一个数的所有真因子（不含自身）之和等于自身，称为完全数。给定N，输出不超过N的所有完全数。

输入 一个整数N。

输出 每行一个完全数。

范围 1 ≤ N ≤ 10⁸

输入	输出
30	6
	28

思路 10^8 以内只有四个完全数：6, 28, 496, 8128。直接判断它们是否 $\leq N$ 即可。不需要枚举计算！这是数学知识在编程中的应用。

技巧 利用数学事实简化计算。C++可以预计算或直接判断。Python同样。

C++

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    int perfect[] = {6, 28, 496, 8128};
    for (int x : perfect)
        if (x <= n) cout << x << endl;
    return 0;
}
```

Python

```
n = int(input())
for x in [6, 28, 496, 8128]:
    if x <= n:
        print(x)
```

NQ054 质数 AcWing 726

给定N，输出2到N之间所有的质数（素数）。质数指大于1且只有1和自身两个因子的数。

输入 一个整数N。

输出 每行一个质数。

范围 $2 \leq N \leq 10^6$

输入

10

输出

2
3
5
7

思路 枚举2到N的每个数，判断是否为质数。优化：只需检查到 $\sqrt{i}$ 。判断函数：for j in range(2, int(sqrt(i))+1): if i%j==0: break。

技巧 $\sqrt{n}$ 优化将复杂度从 $O(n^2)$ 降到 $O(n\sqrt{n})$ 。使用math.isqrt()（Python 3.8+）精确整数平方根。

C++

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    int n;
    cin >> n;
    for (int i = 2; i <= n; i++) {
        bool prime = true;
        for (int j = 2; j * j <= i; j++) {
            if (i % j == 0) { prime = false; break; }
        }
        if (prime) cout << i << endl;
    }
    return 0;
}
```

Python

```
import math
n = int(input())
for i in range(2, n + 1):
    prime = True
    for j in range(2, int(math.sqrt(i)) + 1):
        if i % j == 0:
            prime = False
            break
    if prime:
        print(i)
```